

BIG DATA

CHAPITRE 4:

LANGAGES DE REQUÊTES HADOOP



Ons Ben Rejeb
ISET de Zaghuan
Département TI
Classe 3^{ème} année DSI
Année Universitaire 2020/2021

Plan

RDBMS vs Hadoop

Analyse de données dans Hadoop

Tableau comparatif

Pig ?

Pig : Etapes d'un programme Pig

Pig : Types de données

Pig: Exemple

Exercices

Plan

Hive ?

Hive: Modèles de données

Hive: Exemple

Jaql ?

Jaql : Requête

Jaql : Exécution

Jaql : Exemple

RDBMS vs Hadoop

	RDBMS	HADOOP
Source de données	Structurées	Même non structurées
Type de données	Enregistrement, objet, XML	Fichiers
MAJ de données	MAJ possible	Insertion et suppression
Langage	SQL et XQuery	Pig(PigLatin), Hive(HiveQL),Jaql
Type d'exécution	Temps Réel	En Batch
Sécurité	Sécurité et audit	Partiel
Compression	Sophistiquée	Simple
Matériel	Entreprise	Banalisé
Accès aux données	aléatoire	Fichier seulement
Date création	~40 année	<5 année

RDBMS : Relational Database Management System

Analyse de données dans Hadoop

- Pour éviter la complexité du modèle de programmation de Hadoop quelques langages ont été créés:

- Pig
- Hive
- Jaql
- BigSQL



Tableau comparatif

Caractéristique	Pig	Jaql	Hive	BigSQL
Développé par	Yahoo	IBM	Facebook	IBM
Langage	PigLatin	Jaql	HiveQL	Ansi-SQL
Type de langage	Dataflow	Dataflow	SQL	SQL
Structure de données	Complexe	JSON semi-structurées	Généralement Structurées	Généralement Structurées
Schéma	Optionnel	Optionnel	Obligatoire	Obligatoire

Pig ?

- Plateforme capable de traiter plusieurs types de données
- Deux composantes:
 - Langage PigLatin
 - Runtime Environnement
- Deux modes d'exécution
 - Local: `pig -x local`
 - Distribué: `pig -x mapreduce`

Pig: Etapes d'un programme Pig

- LOAD: charger des données depuis HDFS
- TRANSFORM:
 - Ensemble de tâches map et reduce
 - Opérateurs relationnels (Filter, Foreach, Group, Union, etc.)
- DUMP ou STORE: Affichage sur écran ou enregistrement dans un fichier

Pig: Types de données

- Simples: entier, réel, caractère, etc
- Complexes:
 - Tuple : ensemble de champs ordonnés
(John, 18)
 - Bag : Collection de tuples
{(John, 18), (Mary, 29)}
 - Map : ensemble de paire clé/valeur
[nom#John, Tel#12345678]

Pig: Exemple

- Fichier: wordcount.pig

```
myinput = LOAD './all_web_pages' AS (line:chararray);
-- Extraire les mots de chque ligne et les mettre dans un fichier de type pig
--ensuite aplatir le fichier pour avoir un mot par ligne
words = FOREACH myinput
GENERATE FLATTEN(TOKENIZE(line)) AS word;
-- Créer un groupe pour chaque mot
word_groups = GROUP words BY word;
-- compter le nombre des entrés pour chaque groupe
word_count = FOREACH word_groups
GENERATE COUNT(words) AS count, group;
-- Ordonner les résultats selon le nombre d'itération
ordered_word_count = ORDER word_count BY count DESC;
STORE ordered_word_count INTO './word_count_result';
```

Pig: Exemple

- Exécution

- Mode local

```
/bin/pig -x local wordcount.pig
```

- Mode distribué (MapReduce)

```
hadoop dfs -copyFromLocal all_web_pages input/all_web_pages  
bin/pig -x mapreduce wordcount.pig
```

Hive ?

- Infrastructure Data Warehouse
- Composantes:
 - Shell
 - Pilote
 - Compilateur
 - Moteur d'indexation
 - Métastore (schéma physique)

Hive: Modèles de données

- Table : analogue à une table RDBMS
- Partition : optimisation pour l'accès aux données
(Exemple: par date)
- Bucket : Données dans chaque partition divisées
dans des buckets basés sur des colonnes d'une
table

Hive: Exemple

- Analyse des fréquences des films

- Création d'une table selon le format texte en utilisant le délimiteur de tabulation

```
hive> CREATE TABLE movie_ratings (  
  userid INT,  
  movieid INT,  
  rating INT)  
ROW FORMAT DELIMITED  
FIELDSTERMINATED BY '\t'  
STORED AS TEXTFILE;
```

- Enregistrement des données

```
hive> LOAD DATA INPATH 'hdfs://node/movie_data'  
OVERWRITE INTO TABLE movie_ratings;
```

- Calcul de fréquence par filme

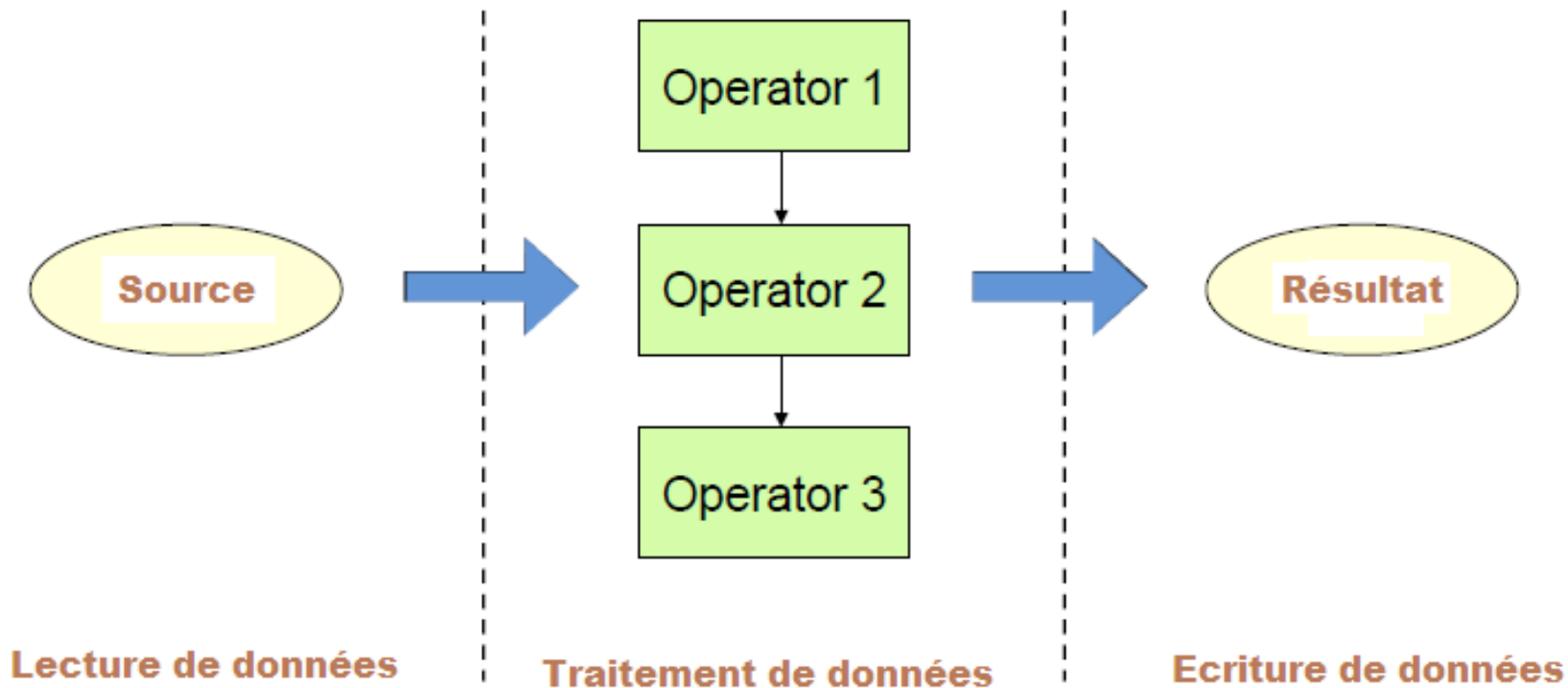
```
hive> SELECT movieid, rating, COUNT(rating)  
FROM movie_ratings  
GROUP BY movieid, rating;
```

Jaql ?

- Manipulation facile
- Analyse de données semi-structurées
- Format : JSON, XML, CSV, Etc.
- Extensible facilement
- Flexible avec des schémas optionnels

Jaql : Requête

- Equivalents à un pipeline
- Opérateurs: FILTER, TRANSFORM, GROUP, JOIN, EXPAND, SORT, TOP



Jaql : Exécution

- Jaql offre des fonctions prédéfinies
- Modèle de données
 - Type atomique : Booléen, Chaine, Etc.
 - Type complexe : Tableau, Enregistrement, Etc.
- Où exécuter les requêtes Jaql
 - Shell: jaqlshell
 - Eclipse
 - Inclus dans Java

Jaql : Exemple

- Chercher les employés qui sont des managers (mgr) ou ayant un salaire (income) >5000

read

Requête

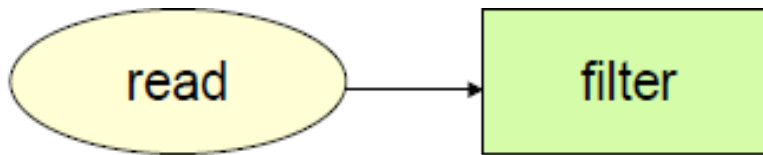
```
read(hdfs "employees" );
```

Données

```
[  
  {name: "Jon Doe", income:  
    40000, mgr: false},  
  {name: "Vince Wayne",  
  
    income: 52500, mgr:  
      false},  
  {name: "Jane Dean",  
  
    income: 72000, mgr:  
      true},  
  {name: "Alex Smith",  
  
    income: 35000, mgr:  
      false}  
]
```

Jaql : Exemple

- Chercher les employés qui sont des managers (mgr) ou ayant un salaire (income) >5000



Requête

```
read(hdfs( "employees" ))  
-> filter $.mgr or $.income > 50000;
```

Données

```
[  
  {name: "Vince Wayne",  
   |income: 52500, mgr:  
   |false},  
  {name: "Jane Dean",  
   |income: 72000, mgr:  
   |true}  
]
```

Jaql : Exemple

- Chercher les employés qui sont des managers (mgr) ou ayant un salaire (income) >5000



Requête

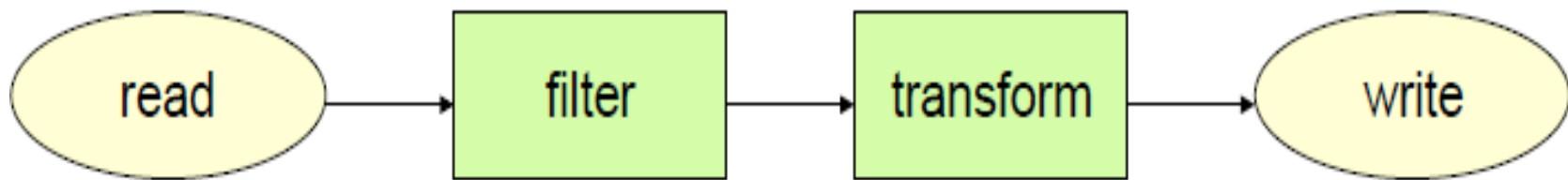
```
read(hdfs( "employees" ))  
-> filter $.mgr or $.income > 50000  
-> transform { $.name, $.income };
```

Données

```
[  
  {name: "Vince Wayne",  
    income: 52500},  
  {name: "Jane Dean",  
    income: 72000}  
]
```

Jaql : Exemple

- Chercher les employés qui sont des managers (mgr) ou ayant un salaire (income) >5000



Requête

```
read(hdfs( "employees" ))  
-> filter $.mgr or $.income > 50000  
-> transform { $.name, $.income }  
-> write(hdfs( "output" ));
```

Données

```
[  
  {name: "Vince Wayne",  
   income: 52500},  
  {name: "Jane Dean",  
   income: 72000}  
]
```

Exercices

▪ Exercice I

Supposons qu'on veut analyser les tickets de caisse d'une grande surface

1) Créer un fichier somme.pig qui:

- charge en mémoire tous les tickets de caisse
- calcul la somme des transactions par article triées par ordre décroissant

2) Donner la commande qui permet d'exécuter ce fichier en local

Exercices

▪ Exercice2

- 1) Créer la table « Article » avec HiveQL en utilisant le délimiteur de tabulation (Article: Id, Désignation, Quantité)
- 2) Charger les données depuis le fichier « ticketsCasse » vers la table article
- 3) Donner la requête qui calcul le total des quantités vendues par article

Exercices

▪ Exercice3

Table Article:

Id_Article	Nom_Articles	Quantité_Articles
1	souris	20
2	clavier	12
3	DD externe	2
4	clé USB	8
5	webcam	65
6	cable ethernet	88
7	imprimante	10

Requête: Donner les articles (id_Article, Nom_Article) ayant une quantité de stock < 10

Question: Illustrer toutes les étapes de Jaql pour exécuter cette requête